

تخلیه بار وظیفه مبتنی بر یادگیری تقویتی در کاربرد ها و برنامه های IoT در پردازش fog: الگوریتم ها

و تکنیک های بهینه سازی.

طاها بیک لریان

دانشگاه علم و صنعت

مقدمه:

در این مقاله مجموعه مقالاتی را که درباره کاربرد های RL و DRL در حل مشکلات و بهینه سازی پردازش ها و محاسبات بر بستر زیرساخت های Fog انجام می شوند و به خصوص در حیطه تخلیه وظایف پردازشی دستگاه های IoT به زیرساخت های Fog ای. پارادایم های پردازشی Fog می توانند در پردازش وظایف و پردازش های سنگین و حساس به زمانی که برای وسایل و استفاده های IoT هستند کمک کنند و آن را تسریع بخشند و باعث بهبود کارایی آنها شوند. از آنجایی که ابزار های IoT برای خیلی از task ها منابع محدودی برای پردازش محلی دارند پس باید task ها را به یک زیرساخت با منابع کافی بفرستند و پاسخ پردازش آن را بگیرند و از آنجایی که زیرساخت های ابری که این دستگاه های IoT، وظایف خود را به به آن ها می فرستند به صورت متمرکز و معمولا خیلی دورتر از محل تولید داده و task هستند، این موضوع عملکرد دستگاه های IoT را، مخصوصا برای وظایفی که حساس به زمان هستند دچار مشکل می کند. برای حل این مشکلات یک پارادایم محاسباتی/پردازشی دیگری به اسم Fog تعریف شده است که منابع محاسباتی و پردازشی و ذخیره سازی داده ها و task ها را بصورت غیر متمرکز در نزدیک محل تولید داده و task و وسایل IoT قرار میدهد. با این کار هم کارایی سیستم IoT را با کاهش تاخیر در پردازش داده؛ با نزدیک کردن منابع پردازشی، افزایش داده است و هم طول عمر باتری وسیله IoT را افزایش داده است؛ با برداشتن بار پردازشی از روی منابع وسیله IoT. حال با علم بر این مطالب، طراحی الگوریتمی که تصمیم بگیرد کدام task را offload کند و کجا آن ها را اجرا کند اهمیت پیدا می کند. با توجه به حجم داده ای که از طریق وسایل IoT تولید میشه و هر ساله این مقدار افزایش می یابد، اینکه در این فضای داده ای بزرگ چه استراتژی ای را برای انجام task ها به کار ببریم که بهترین نتیجه و کارایی را بگیریم اهمیت پیدا میکند. اینکه این حجم زیاد از دیتا می تواند مشکل ساز باشند میتواند به دلایل: پهنای باند محدود، افزایش و در نتیجه افت کارایی در نتیجه این افزایش پردازش ها، کارآمد نبودن انتقال این حجم از داده به محلی دور تر از محل تولید داده، افت کارایی وسیله ای که نیاز به پاسخ بلادرنگ دارد افت می کند در نتیجه به تاخیر زمانی end to end در

انتقال و پردازش و دریافت پاسخ و در نهایت یک سری داده ها می توانند شامل پیش نیاز های امنیتی و حریم شخصی باشد که این موضوع می تواند با قرار گرفتن داده ها در شبکه تا رسیدن به زیرساخت ابری تحدید شود. رایانش Fog می تواند چاره ای برای این مشکلات دستگاه ها و وسایل IoT باشد، که هم از مزایای زیرساختی ابری استفاده کرده و هم از غیر متمرکز بودن زیرساخت های Edge. و در کل تصمیم گیری که برای offload کردن اتفاق می افتد به دلیل این است که مطمئن شویم offload ان task برای آن وسیله IoT فایده ای مضاعف دار یا خیر. متغیر بودن حجم داده ها و task ها، پیچیدگی task ها و کیفیت خدمات رسانی یک وسیله IoT باعث شده است که از ابزار های یادگیری ماشین مخصوصا یادگیری تقویتی برای حل مسائل بهینه سازی در offloading استفاده کنیم. این ابزار برای زمان های که پویایی زیادی در رفتار های یک سیستم ممکن است وجود داشته باشد استفاده می شود، و ابزاری برای مشکل عدم اطمینان محیط است. و همانطور که در مقاله های دیگر هم مشاهده کرده ایم به دلیل وجود فضای حالت بسیار بزرگی که در آن قرار داریم از DL هم برای بهبود عملکرد مدل های یادگیری ژرف استفاده می کنیم.

مروری بر Fog Computing:

رایانش مهی یا Fog Computing یک زیرساخت پردازشی/محاسباتی توزیع شده است که سعی دارد تا قابلیت ها و سرویس های زیرساخت های ابری را به محل تولید داده و وظایف پردازشی، با فراهم کردن منابع پردازشی از قبیل حافظه و پردازشگر و ... به صورت توزیع شده و در نزدیک ایجاد وظایف پردازشی، بیاورد. این یک معماری افقی در سطح سیستم است که عملکرد های محاسباتی، ذخیره سازی، کنترل و شبکه را در امتداد یک پیوستار از ابر به اشیا توزیع می کند. منظور از (معماری افقی) معماری ای است که می تواند سرویس ها و قابلیت هایی برقرار کند که قابل استفاده در همه زمینه ها از جمله صنایع و اپ ها باشد بجای اینکه فقط برای یک استفاده از آن استفاده شود، بصورت (عمودی).

برخلاف رایانش ابری، رایانش Fog را یکسری Fog server و Fog Node تشکیل می دهند که دستگاه هایی در نزدیکی کاربران هستند. که با هدف انجام دادن یکسری وظایف پردازشی در نزدیکی محل تولید آن ها و کاهش زمان پاسخ انجام وظیفه پردازشی است. هر وسیله ای که قدرت پردازشی، شبکه ای و حافظه داشته باشد میتواند بعنوان Fog Node های عمل کنند مانند کنترلر ها، gateway های هوشمند، سویچ ها و روتر ها یا سیستم های embedded. آن ها به طور گسترده قابل در مکان های مختلف قابل استقرار هستند و زمان انتقال داده را با کم کردن فاصله نودهای پردازشگر کاهش می دهند. Fog Node ها به صورت هوشمند داده ها را پردازش، ذخیره می کنند و آن ها را بین زیرساخت های Edge و Fog و Cloud انتقال می دهند.

زیر ساخت های Fog یک ساختار سه لایه دارند:

- لایه IoT یا End layer: در پایین ترین بخش این معماری قرار گرفته است که شامل وسایل IoT ای می شود که داده تولید می کنند و به طبع آن وظایفی پردازشی بر روی آن داده ها باید اعمال شود. از آنجایی که این وسایل قدرت پردازشی بسیار ضعیفی دارند خود به تنهایی نمی توانند همه وظایف پردازشی را انجام دهند و باید مقدار زیادی از آنها را به زیر ساخت های پردازشی دیگری با قدرت پردازشی بهتر انتقال دهند.
- لایه Fog: این لایه یک لایه میانی است و در بین Edge layer و cloud layer قرار دارد که از دستگاه های Fog یا همان Fog Node به صورت ناهمگن که قابلیت های پردازشی محدود، ذخیره سازی محدود و ظرفیت شبکه ای محدوده مانند access point ها، base station ها، router ها، gateway switch ها، و ... تشکیل شده اند. دیوایس های این لایه یک ارتباطی با لایه ابری نیز برقرار دارد که در صورت نیاز در زمان عدم توانایی انجام وظیفه پردازش آن را به لایه بالاتر یا همان لایه ابری و زیرساخت های ابری محول کند. در زیرساخت Fog و لایه Fog دو نوع Fog node خواهیم داشت، High level و Low level، که بر اساس قدرت پردازشی و ذخیره سازی آنها طبقه بندی می شوند.

- لایه Cloud: بالاترین لایه در این معماری است که شامل سرور های ابری و دیتاسنترهای ابری متعددی است که توانایی محاسباتی و پردازشی خیلی بالاتری نسبت به لایه پایینتر که زیرساخت های Fog هستند، ایجاد می کنند.

با توجه به ساختار معماری سیستم های Fog می توان نزدیکی زیرساخت های محاسباتی Fog و Fog Node ها را به دیوایس های IoT و end layer را مشاهده کرد، و به این طریق است که زیر ساخت های Fog می توانند به صورت جغرافیایی توزیع شوند و به صورت غیر متمرکز در بیاید.

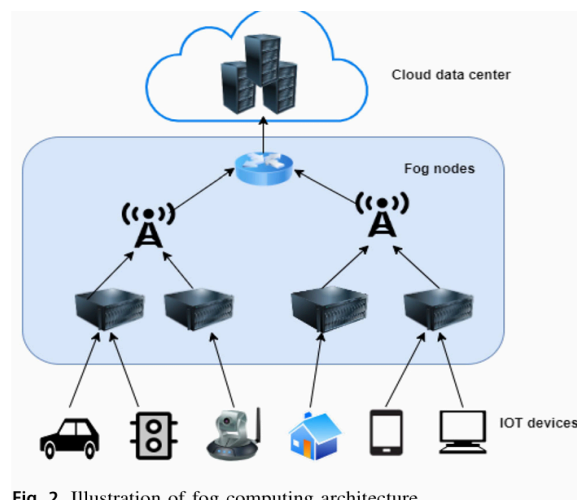


Fig. 2 Illustration of fog computing architecture

معماری Fog Computing:

رایانش Fog یک معماری چند کاره را ارائه می دهد که می تواند برای نیاز های مختلفی به کار بیاید. مدل های مختلفی برای Fog Computing با توجه به محالی که ایجنت ها واقع شده اند وجود دارد مانند: توزیع شده، متمرکز، غیر متمرکز.

- مدل توزیع شده: چندین عامل بر روی سرورهای مه (Fog) مستقر هستند و تصمیمات مربوط به تخلیه بار (offloading) را برای دستگاه های کاربری متصل اتخاذ می کنند. این سرور ها برای عمل کردن به عنوان یک سیستم واحد با یکدیگر ارتباط برقرار می کنند. در این پیکربندی، یک گره (Node) مه جداگانه به یک هدف مشترک کمک می کند، یا هر گره به دستگاه کاربری خود سرویس می دهد و مدیریت منابع را آسان می کند.
- متمرکز: یک سرور مرکزی واحد با قابلیت محاسباتی بالا، تمام تصمیمات مربوط به تخلیه بار را برای وظایف دستگاه ها اتخاذ می کند. این رویکرد در حالی که با متمرکز کردن داده ها امنیت بیشتری را ارائه می دهد، اما کارایی پایین تر و آسیب پذیری بالاتری نسبت به خطا در مقایسه با معماری های توزیع شده یا غیر متمرکز دارد.
- غیر متمرکز: هر سرور در سطوح مختلف، تصمیمات خود را اتخاذ می کند و به صورت جمعی هماهنگ می شوند تا عملکرد کلی سیستم را تعریف کند. این رویکرد تحمل خطای بالا را فراهم می کند و از رخ دادن (SPOF) یا همان Single Point Of Failure جلوگیری می کند اما چالش هایی را در مدیریت کپی ها ایجاد می کند.

ویژگی ها و مزایای کلیدی محاسبات Fog:

- تاخیر کم: نزدیکی Fog Node ها به کاربران دیوایس های Edge، تاخیر انتقال داده را به طور قابل توجهی کاهش می دهد و از برنامه های حساس به زمان مانند ارتباطات وسایل نقلیه هوشمند و واقعیت مجازی پشتیبانی می کند.
- پشتیبانی بالا از تحرکات وسایل: Fog computing منابع محاسباتی و ذخیره سازی را در سراسر شبکه فراهم می کند و تحرک کاربران نهایی را در برنامه هایی مانند شبکه های خودرویی آسان می کند.
- پهنای باند بهتر: فیلتر کردن و تجمیع داده ها به صورت محلی، ترافیک شبکه را کاهش می دهد و اجرای وظایف را تحت پهنای باند محدود سرعت می بخشد.

- مقیاس پذیری: گره های Fog به طور گسترده نزدیک کاربران توزیع شده اند، به مدیریت تعداد زیادی از کاربران نهایی و داده های عظیم از اشیاء ناهمگون IoT کمک می کنند و برنامه ها را مقیاس پذیر و سازگار می سازند.

مشکلات Fog Computing:

- مدیریت منابع: گره های Fog ناهمگن (به این معنا که همه آن ها قابلیت ها یکسان ندارند) با قابلیت های محدود، تخصیص، زمان بندی و اشتراک گذاری منابع را دشوار می سازد.
- حریم خصوصی و امنیت: توسعه استراتژی هایی برای افزایش پایداری سیستم در برابر تهدیداتی مانند نفوذ، دسترسی غیر مجاز و فعالیت های مخرب (مانند حملات DoS) بسیار حیاتی است، زیرا داده ها به گره های Fog بالقوه غیر قابل اعتماد offload می شوند.
- مدیریت انرژی: تعداد زیاد گره های Fog به صورت جغرافیایی توزیع شده اند، منجر به مصرف انرژی بالاتری در مقایسه با cloud computing میشود.
- کیفیت سرویس (QoS): تضمین QoS اغلب شامل trade-off هایی بین معیار هایی مانند اتصال پذیری، قابلیت اطمینان، ظرفیت، تاخیر، بهره وری انرژی و امنیت است که با توجه به نیازمندی هایی که به صورت dynamic وجود دارند برنامه ها پیچیده تر می شوند.
- تخلیه بار (Offloading): مشکلی اصلی این است که تصمیم بگیریم کدام وظایف را از دستگاه های منابع محدود به منابع غنی تر Fog/Edge تخلیه یا offload کنیم، کجا و چگونه، تا تاخیر و مصرف انرژی را به حداقل برسانیم و عمر باتری را افزایش دهیم.
- ناهمگنی/ناهمگونی: ماهیت متنوع دستگاه های IoT از قبیل نوع داده ای که تولید میکنند و انواع مختلفی از وظایف پردازشی که نیاز دارند با قابلیت های متفاوت و زیرساخت شبکه، پیچیدگی مدیریت و انتخاب منابع Fog مناسب را، افزایش می دهد.

- پیچیدگی: مدیریت یک محیط بسیار متنوع با دستگاه‌هایی از تولید کنندگان مختلف و پیکربندی‌های نرم‌افزاری/سخت‌افزاری گوناگون، انتخاب محصولات مناسب برای بهبود عملکرد را چالش برانگیز می‌کند.

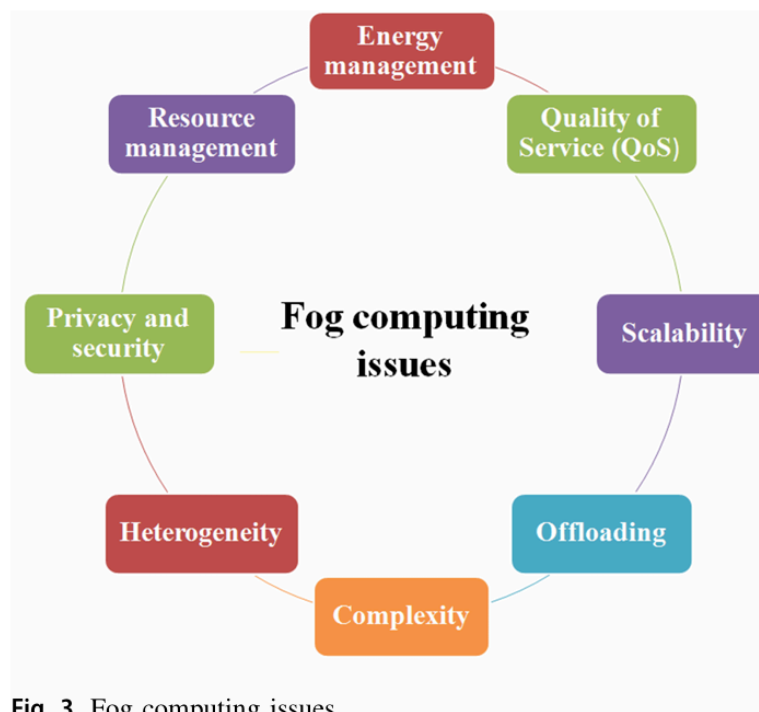


Fig. 3 Fog computing issues

تخلیه وظیفه (Task Offloading):

تخلیه وظیفه یک تکنیک بنیادی در پارادایم fog computing است که برای غلبه بر محدودیت‌های منابع دستگاه‌های اینترنت اشیا طراحی شده است. این تکنیک، امکان انتقال وظایف نیازمند محاسبات فشرده را از این دستگاه‌های دارای منابع محدود به گره‌های Fog یا سرورهای ابری با قابلیت‌های بیشتر فراهم می‌کند. این سازوکار برای پشتیبانی از طیف وسیعی از برنامه‌هایی که نیاز به پردازش در زمان واقعی و مدیریت منابع کارآمد دارند، ضروری است.

هدف اصلی از تخلیه وظایف بهبود عملکرد و کارایی کلی سیستم است. این امر با پیگیری اهداف کلیدی زیر محقق می شود:

- به حداقل رساندن تاخیر اجرا: با انتقال وظایف به منابع قدرتمندتر، زمان کلی مورد نیاز برای تکمیل وظیفه به طور قابل توجهی کاهش می یابد.
 - افزایش عمر باتری دستگاه: تخلیه محاسبات سنگین، مصرف انرژی را در دستگاه های IoT کاهش می دهد و در نتیجه طول عمر عملیاتی آن ها را افزایش می دهد.
 - غلبه بر محدودیت های منابع: این امکان را به دستگاه هایی با قدرت پردازش، حافظه و فضای ذخیره سازی محدود می دهد تا برنامه های پیچیده ای را که در غیر این صورت قادر به انجام آن نیستند، مدیریت کنند.
- یک جنبه حیاتی این فرایند تصمیم برای تخلیه بار است که شامل تعیین اینکه کدام وظایف تخلیه شوند، کجا اجرا گردند و چگونه منابع لازم تخصیص یابند، می شود.
- این مقاله سه نوع اصلی از تصمیمات تخلیه بار را بررسی میکند که بر اساس نحوه مدیریت یک وظیفه توسط سیستم، طبقه بندی می شوند.

- **اجرای محلی (local):** این روش شامل انجام کل محاسبات به طور مستقیم بر روی خود دستگاه اینترنت اشیا است. این رویکرد ترجیحی برای وظایفی است که به شدت حساس به تاخیر هستند و حتی انتقال آن ها به Fog Node ها نیز به صرفه نخواهد بود، یا در سناریویی که دستگاه منابع کافی برای اجرای کارآمد را دارد. همچنین زمانی استفاده می شود که تخلیه بار مقرون به صرفه نباشد یا منابع مورد نیاز آن در نزدیکی آن در دسترس نباشند.
- **تخلیه بار کامل (Full):** این روش شامل انتقال کلی وظیفه از دستگاه محلی به یک سرور راه دور برا پردازش است و از "راه دور" در اینجا منظور ما همان Cloud یا Fog است. پیاده سازی

آن ساده در نظر گرفته می شود و به ویژه برای وظایفی که نمی توانند تقسیم شوند، مناسب تر است، مثل وظایفی که به همدیگر وابستگی دارند و تقسیم آن ها و offload و پردازش آن ها در جاهای مختلف می تواند نا کارآمد باشد و شرایط را دشوار کند. در حالی که این رویکرد صرفه جویی قابل توجهی در مصرف انرژی برای دستگاه محلی به همراه دارد، مهم است که انرژی مصرف شده در طول انتقال داده نیز در نظر گرفته شود.

- **تخلیه بار جزئی(Partial):** این یک رویکرد پویاتر است که در آن یک وظیفه تقسیم می شود، به طوری که بخشی از آن به صورت محلی بر روی دستگاه اجرا می شود و بخش باقی مانده به منابع راه دور برای پردازش فرستاده می شوند. این مدل یک محیط محاسباتی مشترکی بین منابع محلی، Fog و Cloud را ترویج می دهد. تصمیم گیری در دو سطح اتفاق می افتد: ابتدا در دستگاه محلی برای تعیین اینکه کدام بخش ها تخلیه شوند، و سپس در زیرساخت Fog برای تصمیم گیری اینکه کدام یک از وظایف تخلیه شده باید توسط یک Fog Node در مقابل ابر مدیریت شود. این روش به ویژه برای برنامه های در مقیاس بزرگ که هم از منابع محلی و هم از منابع قوی تر دورتر برای بهینه سازی عملکرد استفاده می شود، مزیت دارد.

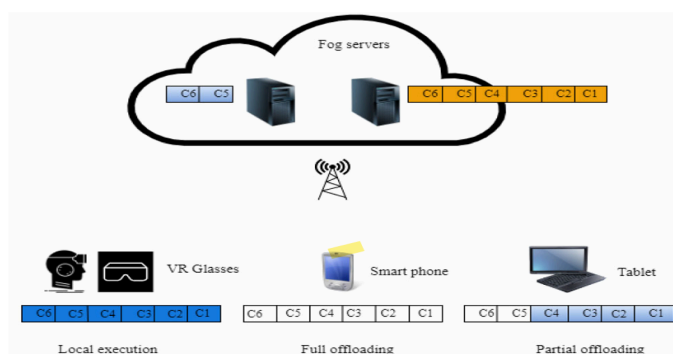


Fig. 4 Outputs of computing offloading decision for different application components at mobile devices

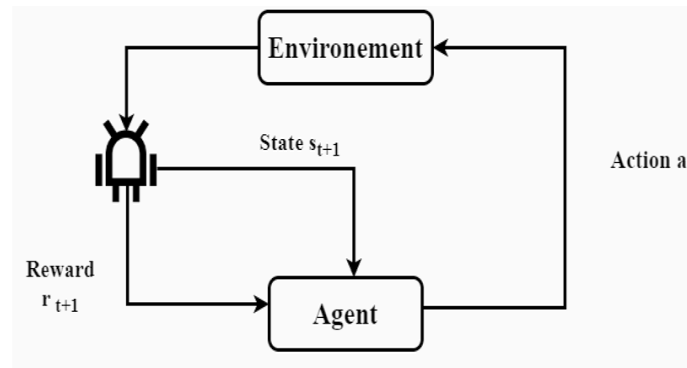
این مقاله اشاره میکند که روش های سنتی تخلیه بار وظایف، مانند nonlinear programming و mixed integer programming و heuristic programming، اغلب برای محیط های dynamic و

پویای Fog کافی نیستند و عملکرد مورد قبول را نخواهند داشت. این روش ها معمولا static هستند و تنوع و پویایی و ناهمگونی task ها را در نظر نمی گیرند. آن ها در اتخاذ تصمیمات real-time و انطباق با ماهیت دائما در حال تغییر شرایط سیستم با مشکل مواجه هستند، که نیاز به راه حل های تطبیق پذیر و هوشمند تر مانند آن هایی که توسط یادگیری تقویتی ارائه می شوند را برجسته می کند.

یادگیری تقویتی:

روش های بهینه سازی سنتی برای تخلیه بار وظایف، مانند non-linear programming و mixed integer programming و heuristic programming برای ماهیت پیچیده و غیر ایستای و پویای Fog computing کافی نیست. این روش ها static هستند و به صورت آفلاین عمل می کنند و قادر به انطباق با تنوع بارهای کاری، تولید پویای وظایف و شرایط متغیر شبکه نیستند. این موضوع منجر به کارآمدی روز افزون مباحث یادگیری تقویتی به عنوان یک پارادایم قدرتمند و مناسب برای رسیدگی به مشکلات تصمیم گیری در real-time مرتبط با task offloading شده است.

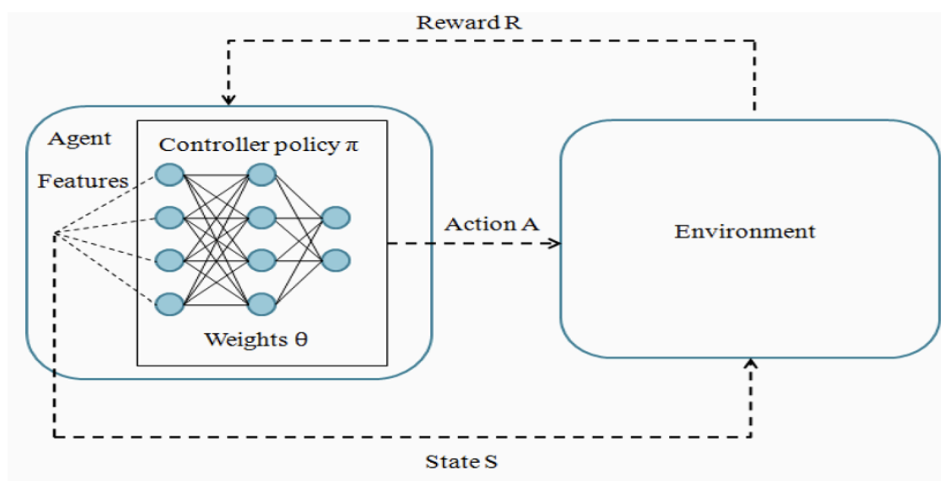
یادگیری تقویتی یک تکنیک یادگیری ماشین است که در آن یک Agent هوشمند یاد می گیرد چگونه با تعامل با یک محیط، تصمیمات بهینه بگیرد. این فرایند یادگیری بر اساس رویکرد آزمون و خطا است و هدف عامل، به حداکثر رساندن پاداش تجمعی است که در طول زمان دریافت می کند. پایه و اساس ریاضی این فرآیند اغلب فضای تصمیم گیری MDP است که اجزای اصلی آن شامل یک Agent که تصمیم گیرنده در محیط است و با محیط تعامل دارد، یک "محیط" است که محیطی ایست که Agent در آن فعالیت می کن. یک State است که توصیفی از وضعیت فعلی محیط است، یک Action است که عملی ایست که عامل در یک state مشخص انجام میدهد و یک پاداش یا reward است که با توجه به مفید بودن یا نبودن عمل انجام شده توسط agent به آن عمل اختصاص داده می شود و یک policy که مجموعه ای از قوانین یا استراتژی هایی است که عامل برای تعیین اینکه کدام عمل را در یک حالت خاص انجام دهد، استفاده می کند.



برای یافتن یک policy بهینه که در مجموع باعث شود بیشترین پاداش را بگیریم و کمترین خطا را انجام دهیم، یک agent از توابع ارزش برای تخمین مطلوبیت یک حالت یا یک عمل استفاده می کند. تابع ارزش state پاداش مورد انتظار از یک حالت خاص را تخمین می زند، در حالی که تابع $Q(S, A)$ پاداش مورد نظر برای انجام یک عمل خاص در یک state خاص را تخمین می زند. با این حال در شبکه های در مقیاس بزرگ با تعداد زیاد دستگاه ها و state های پیچیده و پویا روش های قدیمی و معمول RL با چالش قابل توجهی رو به رو هستند که از تعداد بالای state ها و dynamic بودن محیط می آید که به آن در اصطلاح curse of dimensionality نیز می گویند. روش های قدیمی و مرسوم RL با استفاده از جدول کار می کند و به تعداد state ها ضرب در action ها بخش بندی می شود این جداول و دونه دونه پر می شوند و می توان با توجه به این ساختار متوجه آن شویم که در محیطی به بزرگی و پویایی که در آن زیر ساخت های Fog کار می کنند نگه داشتن و پر کردن این جداول بسیار دشوار و غیر معقول است.

برای غلبه به محدودیت های الگوریتم های معمول RL که با جداول سر و کار دارند، از یادگیری تقویتی عمیق یا DRL استفاده می کنیم که با ترکیب تکنیک های RL با DNN ها پدید آمده اند. از DNN ها به عنوان تخمین زننده تابع های value function و policy function استفاده می شود. به جای ذخیره یک مقدار برای هر جفت state-action در یک جدول، DNN یک تابع تعمیم یافته را یاد می گیرد که می تواند یک مقدار را برای هر حالت و عمل معین پیش بینی کند. این کار agent را قادر می سازد تا محیط

های با ابعاد بالا، پیچیده و پیوسته را مدیریت کند، قابلیتی که برای fog computing بسیار مهم است.



در این جا الگوریتم هایی که برای RL و DRL آورده شده اند model-free هستند به این معنا که اطلاعاتی از محیطی که در آن هستیم از قبل به agent داده نشده و agent خود با درگیر شدن و در معرض محیط قرار گرفتن اطلاعات محیط را استخراج میکند، و این الگوریتم ها به سه دسته تقسیم شده اند: value-based مانند الگوریتم DQN، که تمرکز آن بر تخمین value function است که با استفاده از آن بتوان یک policy بهینه را پیدا کرد. دسته بندی دیگر policy-based است که این الگوریتم ها policy را مستقیم و بدون اتکا به value function بهینه سازی می کند که از الگوریتم های معروف آن میتوان به A2C و A3C یا همان الگوریتم های actor-critic اشاره کرد.

مکانیزم های بر پایه الگوریتم های RL برای task offloading:

تحقیقاتی در زمینه تخلیه بار وظایف یا تخلیه بار محاسباتی در محیط های Fog computing با استفاده از الگوریتم های RL و DRL انجام شده است. تحقیقات بر اساس نو الگوریتم RL استفاده شده، اهداف بهینه سازی، یافته های کلیدی و محدودیت های شناسایی شده، طبقه بندی می شوند.

الگوریتم های policy-based:

این الگوریتم ها به طور مستقیم یک policy را برای تصمیم گیری یاد میگیرند و بهینه می کنند. این مقالات بر استفاده از روش Actor-Critic تاکید دارند که یک Actor برای انتخاب اقدامات و یک Critic برای ارزیابی آن ها هستند.

الگوریتم های A2C/A3C یک رویکرد multi-agent مبتنی بر A2C برای به حداقل رساندن همزمان task offloading latency و بهینه سازی resource allocation مورد بررسی قرار گرفته است. شبیه سازی ها کاهش قابل توجهی در تاخیر را نشان دادند. به طوری که یک مطالعه به کاهش 80 درصدی رسیده است. با این حال مشکل این رویکرد scalable نبودن آن است و در زمانی که تعداد edge device ها و درخواست ها و computation ها زیاد میشوند هزینه های latency افزایش می یابد. یک ضعف کلیدی شناسایی شده، نادیده گرفتن مصرف انرژی به عنوان یکی از اهداف بهینه سازی و عدم مقایسه با سایر الگوریتم های مشابه RL بود. مطالعه های دیگری نیز در این زمینه ها برای بهینه سازی به کمک این الگوریتم ها انجام شده اما آنها نیز مشکل scalable نبودن دارند.

الگوریتم های value-based:

این الگوریتم ها بر یادگیری و تخمین value function برای تعیین بهترین action ها تمرکز می کنند. در این جا به جزئیات چند مطالعه با استفاده از Q-Learning و انواع یادگیری های عمیق می پردازیم.

الگوریتم SARSA یک الگوریتم on-policy است به این معنا که تابع exploration با تابع exploitation در policy متفاوت نیست و هر دو یک تابع هستند برخلاف متد های off-policy مانند Q-learning. الگوریتم SARSA با منطق فازی ترکیب شده تا مصرف انرژی را در محیط Fog خودرویی به حداقل برساند. این رویکرد با موفقیت مصرف انرژی را بیش از 48 درصد در مقایسه با روش های سنتی کاهش داد. با این حال، یک نقطه ضعف قابل توجه دارد، عملکرد نه چندان بهینه آن از نظر

تاخیر است که در نتیجه منجر به زمان سرویس کلی بالاتر می شود. این مقاله و خیلی از مقاله های دیگر مسائل امنیتی داده های offload شده را نادیده می گیرند و مانند ایراد های دیگری که این مقاله دارد مانند خیلی از مقالات بررسی شده در طول این مقاله این است که الگوریتم های RL دیگری را مقایسه نکرده.

الگوریتم های Q-Learning و DQN: با استفاده از یک مدلی که از ترکیب LSTM با Q-Learning بدست می آید، نشان داد که به طور موثر زمان اجرا، مصرف انرژی و استفاده از CPU را متعادل می کند و به کاهش قابل توجهی در هر متریک دست میابد. در همین حال، پرکاربردترین الگوریتم، DQN، در چندین مقاله دیگر نیز به کار گرفته شده. از جمله آنها در یک مقاله ای یک رویکرد مبتنی بر DQN با موفقیت تاخیر اجرا را به میزان قابل توجهی کاهش داد اما به دلیل استفاده از مدل single-agent قابل استناد نیست به دلیل اینکه میتواند با مشکلات scalable نبودن مواجه شد. یا مقاله ای دیگری از contract theory و روش مبتنی بر DQN برای افزایش QoS در کاربرد های خودرویی استفاده کرد اما مسائل سیال و متحرک بودن آنها را در نظر نگرفته!!!!

الگوریتم های DRQN و Dueling DRQN به عنوان انواع DQN بررسی شده اند. یک روش مبتنی بر DRQN که بر روی معماری SDN عمل می کرد، در به حداقل رساندن میانگین تاخیر وظیفه موثر بوده اما معایبی از قبیل امنیت و scalable نبودن خود SDN آن را محدود کرده است. یک مدل Dueling DQN، که برای تخلیه بار multi-agent و multi-task طراحی شده بود، کمترین تاخیر و دومین مصرف انرژی پایین را در بین مقالات دیگر نشان داد، اگر چه به دلیل تمرکز بر هدف واحد و عدم اعتبار سنجی در دنیای واقعی قابل استناد نیست.

الگوریتم های Hybrid:

رویکردهای ترکیبی نقاط قوت هر دو الگوریتم policy-based و value-based را ترکیب می کند.

الگوریتم های DDPG و SAC از این الگوریتم های هستند، که این الگوریتم ها برای بهینه سازی کارایی سیستم و QoE استفاده شده اند. یک چهارچوب multi-agent مبتنی بر DDPG با موفقیت کارایی سیستم را بهبود بخشید اما مشکلاتی از قبیل scalable نبودن دارد. یک نوع دیگر DDPG بر مصرف انرژی تمرکز داشت اما تاخیر را نادیده می گرفت. مطالعات دیگر با DDPG و SAC برای برآوردن الزامات QoS و مهلت های وظیفه استفاده کردند که موثر بودن آن ها را ثابت کرده اما اغلب آن ها اعتبار سنجی دقیق و درستی در محیط های پویا یا دینامیک ندارند.

بحث تحلیلی و خلاصه:

تحلیل جامع چندین روند و یافته کلیدی را در تحقیقات این مقاله نشان داده اند که DQN محبوبترین الگوریتم بین پژوهشگران است که در 35 درصد مقالات استفاده شده است و پس از آن DDPG با 24 درصد قرار دارد. این نشان می دهد که این الگوریتم ها برای حل مشکلات computation offloading بسیار موثر تر از باقی الگوریتم ها واقع شده اند.

در مقالات بیشتر از همه سعی بر بهینه سازی latency یا تاخیر شده بود و بعد از آن energy consumption یا همان مصرف انرژی و بعد از QoS QoE شده بود.

بررسی ها نشان می دهد که به دلیل ماهیت پیچیده و پویای سیستم های fog computing، روش های مبتنی بر RL و DRL به عنوان ابزار هایی قدرتمند برای حل مشکلات حیاتی task offloading بوجود آمده اند.

به طور خلاصه، RL و DRL پتانسیل خود را برای تصمیم گیری های هوشمندانه در تخلیه بار اثبات کرده اند، اما یک شکاف قابل توجهی بین عملکرد نظری و کاربرد در دنیای واقعی وجود دارد.

با توجه به مقالات بررسی شده و وضعیت فعلی تحقیقات این مقالات چندین چالش کلیدی را شناسایی کرده اند و مسیر تحقیقاتی آینده را برای تسریر توسعه مکانیسم های task offloading مبتنی بر RL پیشنهاد میکنند:

بهینه سازی joint optimization و ارزیابی در دنیای واقعی و پیشرفت امنیتی و پرداختن به تحرکات وسایل و

با پرداختن به این مسائل تحقیقات آینده می توانند شکاف بین راه حل های نظری و پیاده سازی عملی را پر کند و در نهایت به روش های مبتنی بر RL امکان دهد تا پتانسیل خود را در ساخت اکوسیستم های fog computing قوی تر و ایمن تر و کارآمدتر کنند.

پایان